

1. Rekurzív függvény – faktoriális

```
#include <iostream>
using namespace std;
double fakt(int x);
int main()
{
    int szam;
    cout<<"\nAdjon meg egy 0 -nál nagyobb szamot szamot\n";
    cin>>szam;
    cout<<"A szam faktorialisa : "<<fakt(szam)<<endl;
    return 0;
}
double fakt(int x )
{
    double szorzat;
    if (x == 0) szorzat = 1;
    else szorzat = x * fakt(x - 1);
    return szorzat;
}
```

2. Tömb összege rekurzívan

```
#include <iostream>
using namespace std;
int osszeg(int* tomb, int n);
int main(){
    int t[] = { 1, 2, 3, 4, 5 };
    cout << osszeg(t, 5); // 15
    return 0;
}
int osszeg(int *tomb, int n)
{
    if (n == 0) return 0;      // alapeset
    return tomb[n - 1] + osszeg(tomb, n - 1);
}
```

3. Legnagyobb közös osztó:

```
#include <iostream>
using namespace std;
int lnko(int x, int y);
int main()
{
    int szam1, szam2;
    cout<<"Adja meg a ket szamot";
    cin>>szam1>>szam2;
    cout<<"\nA ket szam legnagyobb kozos osztója"<<lnko(szam1,szam2)<<endl;
    system("pause");
    return 0;
}
int lnko(int x, int y)
{
    int max;
    if (x==y) return x;
```

```

    if(x<y)max=lnko(x,y-x);
        else max=lnko(x-y,y);
    return max;
}

/*Függvény lnko(a, b : egész számok) : egész szám
ha a=b akkor lnko := a
ha a<b akkor lnko := lnko(a, b-a)
ha a>b akkor lnko := lnko(a-b, b)
*/

```

4. Hanoi tornyai probléma megoldása rekurzívan. Kiinduló helyzet => minden korong (tetszőleges számú) az első rúdon, alul a legnagyobb felette egyre kisebb korongok. Cél: helyezzük az összes korongot a 3-as rúdra a segéd rúd (2) segítségével de kisebb korongra nagyobb nem kerülhet, Számoljuk a lépések számát is!

```

#include <iostream>
using namespace std;
int hanoi(int n, char honnan, char segedrúd, char hova)
{
    static int db; // számolja hány lépésből áll a megoldás
    if( n>0)
    {
        hanoi(n-1, honnan, hova, segedrúd); //n-1 db korong átmozgatása a "segedrúd"-re,
        cout<<n<<" . korongot tedd a "<<honnan<<"rudra a "<<hova<<" rudra! \n";
        hanoi(n-1, segedrúd, honnan, hova); //n-1 db korong átmozgatása a "segedrúd"-ről a "hova" rudra
        db++;
    }

    return db;
}

int main()
{
    int korong;
    cout<<" adja meg a korongok szamat: \t";
    cin>>korong;
    cout<<"\n\n"<<hanoi(korong,'1','2','3')<<" lepesben lehetett athelyezni a korongokat\n"; //
    kiinduló helyzet minden korong az 1-es rudon
    return 0;
}

```

5. Tesztverseny (függvénnnyel)

Egy közismereti versenyen a versenyzőknek 14 tesztfeladatot tűznek ki. A versenyzőknek minden feladat esetén négy megadott lehetőség (A, B, C, D) közül kell a helyes választ megjelölniük. A versenybizottság garantálja, hogy tesztlapon minden kérdéshez pontosan egy helyes válasz tartozik. A kitöltött tesztlapokat elektronikusan rögzítik, a visszaélések elkerülése végett a versenyzőket betűkből és számokból álló kóddal azonosítják.

A helyes megoldást és a versenyzők válaszait a *valaszok.txt* szöveges állomány tartalmazza. A fájlban legfeljebb 500 versenyző adatai szerepelnek. A fájl első sorában a helyes válaszok szerepelnek. A fájl többi sora a versenyzők kódjával kezdődik, ezt egy szóköz, majd az adott versenyző által adott válaszok sorozata követi. A versenyzők kódja legfeljebb 5 karakterből áll.

A válaszok a feladatokkal egyező sorrendben, elválasztójel nélkül, nagybetűvel szerepelnek. Ha a versenyző egy kérdésre nem válaszolt, akkor annak helyén X betű szerepel.

BCCCDBBBBBCDAAA

helyes válaszok

AB123 BXCDBBACACADBC

1. versenyző kódja és adatai

AH97 BCACDBDDBCBBCA

Feladatok:

1. Olvassa be és tárolja el a *valaszok.txt* szöveges állomány adatait (a helyes választ és a versenyzők eredményeit) egy megfelelő adatszerkezetbe, tárolja az egyes versenyzők pontszámait is (minden helyes válasz egy pont)!
2. Írja ki a képernyőre, hogy hány versenyző vett részt a versenyen!
3. Írja ki a legtöbb pontot elért versenyző azonosítóját és pontszámát!
4. Döntse el volt e maximális pontszámot elért versenyző!
5. Számolja meg hány versenyző nem tudott legalább a kérdések felére válaszolni!
6. Rendezze a pontszám szerint csökkenő en az eredményeket és írja ki az első 10 helyezettet a képernyőre!

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
struct Teszt
{
    string azonosito;
    string valasz;
    int pont;
};
int Adatbe(Teszt* tmb, string filenev);
int Legtobbpont(Teszt* tmb, int db);
bool Voltemax(Teszt* tmb, int db);
int Hetalatt(Teszt* tmb, int db);
void Elsotiz(Teszt* tmb, int db);

int main()
{
    // 1_2. Adatbeolvasás + Írja ki a képernyőre, hogy hány versenyző vett részt a versenyen!
    Teszt tomb[500];
    string fnev = "valaszok.txt";
    int versenyzokszama = Adatbe(tomb,fnev);
    cout << "A versenyen " << versenyzokszama << " versenyzo vett részt" << endl;

    //3.Írja ki a legtöbb pontot elért versenyző azonosítóját és pontszámát!
    int legjobb = Legtobbpont(tomb, versenyzokszama);
    cout<<"A legtöbb pontot elert versenyzo azonositoja : "<< tomb[legjobb].azonosito<<"
    pontszama: "<< tomb[legjobb].pont<<endl;

    //4.Döntse el volt e maximális pontszámot elért versenyző!
    if (Voltemax(tomb, versenyzokszama)) cout<<"Volt maximalis pont"<<endl;
    else cout<<"Nem volt maximalis pont"<<endl;
```

```

//5.Számolja meg hány versenyző nem tudott legalább a kérdések felére válaszolni!
cout<< Hetalatt(tomb, versenyzokszama)<<" versenyzo nem tudott legalabb a kerdések
felere valaszolni"<<endl;
// 6.Rendezze a pontszám szerint csökkenő en az eredményeket és írja ki az első 10
helyezett a képernyőre!
Elsotiz(tomb, versenyzokszama);
return 0;
}
int Adatbe(Teszt* tmb, string filenev)
{
    ifstream be("valaszok.txt");
    if (be.fail()) { cout << "hiba"; system("pause"); exit(1); }
    int i = 0, j;
    string megoldas;
    be >> megoldas;
    while (be >> tmb[i].azonosito >> tmb[i].valasz)
    {
        int pontsz = 0;
        for (int j = 0; j < 14; j++)
        {
            if (tmb[i].valasz[j] == megoldas[j])
            {
                pontsz++;
            }
        }
        tmb[i].pont = pontsz;
        i++;
    }
    be.close();
    return i;
}
int Legtobbpont(Teszt* tmb, int db)
{
    int max = 0;
    for (int i = 1; i < db; i++)
    {
        if (tmb[i].pont > tmb[max].pont)
        {
            max = i;
        }
    }
    return max;
}
bool Voltemax(Teszt* tmb, int db)
{
    int i = 0;
    while (i < db && tmb[i].pont != 14)
    {
        i++;
    }
    if (i < db) return true;
    else return false;
}

```

```

}
int Hetalatt(Teszt* tmb, int db)
{
    int hetalatt = 0;
    for (int i = 0; i < db; i++)
    {
        if (tmb[i].pont < 7) hetalatt++;
    }
    return hetalatt;
}
void ElsoTiz(Teszt* tmb, int db)
{
    for (int i = 0; i < db - 1; i++) // rendezés közvetlen kiálasztással
    {
        for (int j = i + 1; j < db; j++)
        {
            if (tmb[i].pont < tmb[j].pont)
            {
                Teszt seged = tmb[i];
                tmb[i] = tmb[j];
                tmb[j] = seged;
            }
        }
    }
    // Első tíz
    for (int i = 0; i < 10; i++)
    {
        cout << tmb[i].azonosito << " " << tmb[i].pont << endl;
    }
}

```